

TripBuddy

여행 준비부터 일정 변경까지,
함께 계획하는 AI 여행 플랫폼



기획 배경

여행을 다니며 느낀 불편함

항공권 · 숙소 비교

예약 앱을 오가고

갈 곳 · 이동시간 확인

지도와 후기를 다시 열고

전체 경비 계산

하나씩 더해 봅니다

이 모든 준비를 한곳에서 할 수 있을까?



여행 준비의 불편함을 확인하기 위해 공개 조사와 사용자 경험을 살펴봤습니다

한국 Z세대

80%

여행 예약 과정이 부담스럽다

Skyscanner · OnePoll 조사

검색하고, 확인하고, 연결하는 데 시간과 수고가 듭니다.

일정을 만들기 위해 반복하는 일

정보 수집	관광지 · 식당 · 숙소를 따로 검색
-------	----------------------

조건 확인	영업시간 · 휴무 · 예약 여부 확인
-------	----------------------

동선 구성	방문 순서와 이동시간을 조율
-------	-----------------

힘들게 일정을 짜도, 변수 하나가 생기면 다시 조율해야 합니다

계획 완료

2박 3일 일정

항공편 · 숙소 · 식당
방문 순서까지 정했는데



변수 발생

식당 휴무

날씨 악화

항공편 취소



다시 조율

대체 장소 찾기

방문 순서 변경

이동시간 재계산

한 곳의 변경이, 남은 일정 전체를 흔듭니다.

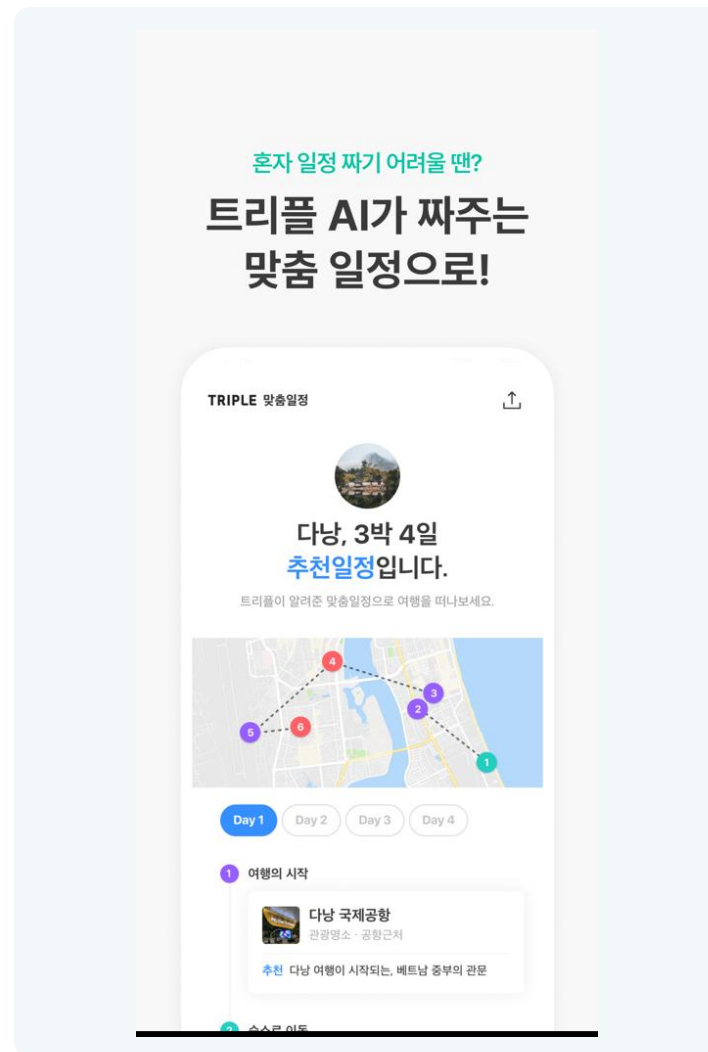
여행 상품은 쉽게 찾습니다. AI 일정은 어디까지 도와줄까요?

기존 플랫폼의 강점

항공·숙소 가격 비교와 투어·액티비티 탐색

파악한 약점 및 미흡한 점

생성은 가능해도 바로 쓰기 어려운 일정
변경 뒤 후속 조정은 여전히 사용자 몫



AI가 짜준 일정, 그대로 여행하기엔 부족했습니다

트리플

장소를 넣고 날짜를 바꾸는 것부터 불편

앱 스토어 이용 후기 · 등록되지 않은 장소와 날짜별 일정 편집을 지적

트립닷컴

시작 시각 하나도 일정판에서 바로 못 고침

2026년 개별 후기 · AI 프롬프트로는 설정 가능하나 일정 화면에서 수정에 제약

생성 이후의 수정과 후속 조치는 사용자 몫으로 남았습니다



여행 준비부터 현지 일정 변경까지
한곳에서 이어갑니다

올인원 AI 여행 플랫폼

TripBuddy

01

FRONTEND

프론트엔드

이시우

01

USER EXPERIENCE

서비스 시연 / 제주 2박 3일

제주 2박 3일 시나리오로 저희 플랫폼을 소개해드리겠습니다



3명의 여행자

항공 + 렌터카
애월 카페 거리 중심의 일정

조건 선택부터 생성·수정까지 시연합니다

제주 2박 3일, 직접 보여드리겠습니다



[로그아웃](#)[내 예약](#)[여행 만들기 →](#)

제주 2박 3일 시나리오

서비스 소개

YOUR BUDGET, YOUR ROUTE

여행의 모든 순간을 함께하는 AI, TripBuddy.

예산과 취향만 알려주세요.
복잡한 준비부터 현장의 변수까지, AI가 여행의 균형을 맞춰드립니다.

[예산 우선 설계](#)[취향 기반 추천](#)[남겨 있는 옵션](#)



AI 일정 설계
조건만 고르면 일정 완성



항공권 비교
출발지와 날짜만 비교



숙소 찾기
취향에 맞는 숙소 한눈에



투어 액티비티
내셔널파크부터 로컬 체험까지



교통 렌터카
KTX부터 렌터카까지 한 화면



유심 eSIM
데이터 용량 없는 여행 준비

TRIP PLANNING, MADE PERSONAL

내 예산 안에서 어디로 떠날까요?

여행의 조건을 알려주면 AI가 비용을 먼저 계산하고, 그 안에서 가장 좋은 하루를 찾아드려요.

AI 일정 설계

여행 조건과 예산을 직접 설정해 맞춤 일정을 만들어요.

설정 보기

TripBuddy

02

BACKEND

백엔드

서정찬

02

DATA & LOGIC

실제 여행 데이터, 어디서 가져올 수 있을까?

2개월 프로젝트에서 플랫폼·항공사와 직접 제휴해 원천 데이터를 확보하기는 어려웠습니다

필요한 데이터

숙소 · 항공편 · 장소 · 이동시간

실제 정보가 있어야 여행 일정을 만들고
시간·동선·경비까지 검증할 수 있습니다.

제휴의 장벽



예약·상품의 원천 정보는 제휴가 필요해
단기 프로젝트에서 바로 접근하기 어려웠습니다.

우리의 과제

제휴 없이도 실제 데이터를 바탕으로 작동하는 일정을 만드는 방법 찾기

제휴 데이터의 제약을 다양한 실제 데이터로 보완합니다

제휴사 직접 연계의 한계 → 공공데이터 · 외부 API · 장소 정보 수집

공공데이터 API

행정안전부 · 공공데이터포털



숙박업 인허가 · 영업 상태

운항 API

AeroDataBox



공항 · 항공편 · 운항 시각

경로 API

Kakao Mobility



도로 경로 · 거리 · 이동시간

관광정보 API

VISIT JEJU

제주 관광지 · 장소 상세정보

날씨 API

기상청

단기 · 중기 예보 · 강수 · 기온

유가 정보 API

Opinet

지역별 유가 · 유류비 산정



수집 · 검증 스크래핑

네이버 플레이스

숙소 정보 대조

음식점 카페 스크래핑

카카오맵

음식점 · 카페 장소 ID · 좌표

AI 추론

Amazon Bedrock

선별한 후보로 일정 구성



여행 조건에서 AI 추천까지



서버 입력 구성

항공·숙소 기준 확정
취향·날씨로 후보 선별
필수 목적지와 ID 전달



Bedrock 조합

선별된 장소 안에서
날짜별 방문 순서 제안



서버 후처리

ID·중복·누락 검증
경로와 시간 계산
사용 가능한 일정 구성

서버가 입력과 출력을 검증하고, AI는 후보 안에서 추천

원천데이터 수집과 실제 스크래핑 검증

원천데이터 규모

63,020

숙박업 원천

1,335

관광지

621

음식점

1,304

카페

DB 적재 확인

MariaDB · travel · SQL Editor	
<pre>SELECT 'accommodation_business' source_table, COUNT(*) row_count FROM accommodation_business UNION ALL SELECT 'tourist_attraction', COUNT(*) FROM tourist_attraction UNION ALL SELECT 'restaurants', COUNT(*) FROM restaurants UNION ALL SELECT 'cafes', COUNT(*) FROM cafes;</pre>	
source_table	row_count
accommodation_business	63,020
tourist_attraction	1,335
restaurants	621
cafes	1,304

실제 스크래핑 실행 화면

The image is a composite of three parts. The top part shows a web browser window displaying a Naver hotel search page for Jeju City. The middle part shows a terminal window with a list of hotel details, including ID, rating, star, price, and check-in/out times. The bottom part shows a terminal window with the execution of a scraping script, displaying the number of items saved and the file path.

```
[NAVER] page 24[main]: clicked number text= 24 class=Pageination page_yCtZa
[NAVER] page 24[main]: from=460 structured-items=16 total-unique=476 +16
[NAVER] structured unique accommodations=476
[001] id=1188703706 다인오세아노 호텔 rating=8.7 star=4 price=100,326원 check=15:00/11:00
facilities=46
[002] id=37116745 탈라스테이호텔 rating=7.8 star=4 price=56,888원 check=15:00/11:00 facili
ties=13
[003] id=36281072 유니호텔 제주 rating=8.5 star=4 price=71,747원 check=15:00/11:00 facilit
ies=28
[004] id=16870181 탐스텐빌라드 매월 rating=9.0 star=3 price=88,095원 check=15:00/11:00 fac
ilities=24
[005] id=1604588319 스탠포드호텔앤리조트 제주 rating=8.7 star=3 price=117,853원 check=15:0
0/11:00 facilities=26
[006] id=1016282016 매월스테이 인 제주 rating=8.8 star=4 price=66,370원 check=15:00/11:00
facilities=23
[007] id=1064005604 누에베 플빌라&리조트 rating=5.3 star=None price=99,798원 check=15:00/1
1:00 facilities=33
[008] id=1504775582 코니오션스위트 호텔 rating=8.7 star=2 price=55,534원 check=16:00/11:30
facilities=11
[009] id=1269358860 마레보비치호텔 rating=8.9 star=3 price=66,099원 check=16:00/11:00 faci
lities=18
[010] id=36379072 베니카아호텔제주 rating=7.8 star=3 price=44,644원 check=15:00/11:00 faci
lities=26
[011] id=12884668 다인리조트 rating=8.3 star=3 price=55,629원 check=15:00/11:00 facilities
=30
[012] id=37101956 씨스테이 호텔 rating=8.6 star=3 price=63,661원 check=15:00/11:00 facilit
ies=19
[013] id=13316587 제주도게스트하우스파티 꿀길 365 rating=8.2 star=3 price=66,370원 check=15
:00/11:00 facilities=4
[014] id=1780850920 코론호텔 rating=8.6 star=2 price=50,278원 check=16:00/11:00 facilities
=17
[015] id=37917639 셀리스제주호텔 rating=8.1 star=3 price=61,019원 check=15:00/11:00 facili
ties=13
[016] id=1367080954 스테이 온 rating=None star=2 price=288,505원 check=None/None facilitie
s=0
[017] id=37550280 힌수업고래리조트 rating=9.1 star=3 price=47,420원 check=15:00/11:00 faci
lities=16
[018] id=1604784907 매월 오션스테이 rating=9.1 star=None price=48,761원 check=15:00/11:00
facilities=24
[019] id=1388302899 제주매월애 독채펜션 rating=9.0 star=4 price=81,269원 check=16:00/11:00
facilities=12
[020] id=1317571866 히든힐호텔 rating=8.0 star=3 price=98,241원 check=None/None facilities
=0
[HOTELS-CACHE] saved 476 items -> debug\naver-hotels-cache.json
```

공공데이터 수집부터 스크래핑 검증과 DB 적재까지 연결

실행 화면과 DB 조회 결과를 함께 제시해 수집 과정과 적재 결과를 증명

기능 단위로 백엔드 API를 문서화했습니다

전체 구현 범위와 인증이 필요한 엔드포인트를 그룹별로 확인했습니다

 Swagger
OPENAPI SPECIFICATION

/v3/api-docs

Explore

TripBuddy 여행 서비스 API

1.0.0 OAS 3.1

/v3/api-docs

여행 검색, 추천, 일정 생성·편집, 경로 및 사용자 인증을 제공하는 REST API 명세입니다. 자동화가 표시된 API는 로그인 후 발급받은 JWT 액세스 토큰을 우측 상단 Authorize에 입력해야 합니다.

Servers

http://localhost:8080 - Generated server url

Authorize

관광지

관광지 상세 정보, 검색 결과 및 여행 취향 기반 추천을 제공합니다.

GET

/api/attractions/{attractionsId}

관광지 상세 조회

POST

/api/attractions/search

관광지 검색

POST

/api/attractions/recommend

관광지 추천

날씨

여행지 좌표와 여행 정보를 기준으로 단기·중기 일별 날씨를 제공합니다.

GET

/api/weather/calendar

여행지 일별 날씨 조회

사용자

사용자 관련 API

POST

/api/users/signup

회원가입

POST

/api/users/refresh

엑세스 토큰 갱신

POST

/api/users/logout

로그아웃

POST

/api/users/login

로그인

상태 확인

서버의 상태가 정상적인지 확인합니다.

GET

/health

서버 상태 확인

숙소

여행 지역의 숙소를 검색하고 사용자 조건에 맞는 숙소를 추천합니다.

POST

/api/accommodations/search

숙소 검색

POST

/api/accommodations/recommend

숙소 추천

숙소 데이터 관리

관리자가 공공 숙소 데이터를 초기 적재하거나 갱신할 때 사용하는 API입니다.

POST

/api/admin/accommodations/bootstrap

숙소 데이터 초기 적재

여행

로그인 사용자의 여행을 생성·조회하고 전체 예상 비용을 확인합니다.

GET

/api/trips

내 여행 목록 조회

GET

/api/trips/{tripId}

여행 상세 조회

GET

/api/trips/{tripId}/cost

여행 비용 조회

POST

/api/trips

여행 생성

여행 날씨

저장된 여행의 목적지와 날짜를 기준으로 일지별 예보를 조회합니다.

GET

/api/trips/{tripId}/weather

여행 일정 날씨 조회

여행 일정 편집

생성된 일정의 장소 변경과 순서 변경을 검증하고 실제 이동시간을 다시 계산해 저장합니다.

GET

/api/trips/{tripId}/plan/editor

일정 편집 정보 조회

GET

/api/trips/{tripId}/plan/places

교체 장소 검색

PUT

/api/trips/{tripId}/plan/editor

변경 일정 저장

서비스 기능별 API 구성

관광지 · 날씨 · 사용자 인증 · 숙소 · 여행 · 비용 · 일정 편집

추천 · 경로 · AI 일정 API를 하나의 흐름으로 연결했습니다

도메인별 API가 최종 일정 생성과 편집 과정에 연결되는 구성을 확인했습니다

음식점 음식점 상세 정보, 검색 결과 및 음식 취향 기반 추천을 제공합니다.

GET

/api/restaurants/{restaurantId}

음식점 상세 조회

🔒

▼

POST

/api/restaurants/search

음식점 검색

🔒

▼

POST

/api/restaurants/recommend

음식점 추천

🔒

▼

이동 경로 두 장소의 좌표를 기준으로 자동차 이동 거리, 시간 및 지도 경로를 계산합니다.

POST

/api/routes/driving

자동차 경로 계산

🔒

▼

일정 이동 구간 여행 일자별 장소 사이 이동 구간을 생성·조회·수정·정렬·삭제합니다.

DELETE

/api/trips/{tripId}/days/{dayId}/transport-segments/{segmentId}

이동 구간 삭제

🔒

▼

GET

/api/trips/{tripId}/days/{dayId}/transport-segments

이동 구간 목록 조회

🔒

▼

PATCH

/api/trips/{tripId}/days/{dayId}/transport-segments/{segmentId}

이동 구간 수정

🔒

▼

PATCH

/api/trips/{tripId}/days/{dayId}/transport-segments/reorder

이동 구간 순서 변경

🔒

▼

POST

/api/trips/{tripId}/days/{dayId}/transport-segments

이동 구간 생성

🔒

▼

카페 카페 상세 정보, 검색 결과 및 여행 조건 기반 추천을 제공합니다.

GET

/api/cafes/{cafeId}

카페 상세 조회

🔒

▼

POST

/api/cafes/search

카페 검색

🔒

▼

POST

/api/cafes/recommend

카페 추천

🔒

▼

항공편 출발·도착 공항과 여행 날짜를 기준으로 실제 운항된 및 예상 운항을 조회합니다.

POST

/api/flights/search

항공편 검색

🔒

▼

AI 여행 일정 SQS Worker에 AI 일정 생성을 요청하고 비동기 처리 상태와 완성된 일정을 조회합니다.

GET

/api/trips/{tripId}/plan

AI 일정 생성 상태 조회

🔒

▼

POST

/api/trips/{tripId}/plan

AI 일정 생성 요청

🔒

▼

AI 연결 확인 Amazon Bedrock 모델 연결 상태를 점검하는 개발·운영 확인용 API입니다.

POST

/api/ai/test

AI 응답 테스트

🔒

▼

백엔드 핵심 구현

데이터 수집 → AI 생성 → 검증 → 실제 경로·비용 계산

추천부터 일정 생성까지 연결

음식점 · 이동 경로 · 카페 · 항공편 · AI 일정 생성

조건으로 후보를 줄이고 가중치로 순위를 계산합니다

숙소

30분 이내 후보 우선

부족하면 45분 → 전체
도로 접근성 35%
베이지안 평점 65%

관광지

서버가 30~50개로 압축

대표성·선호 각 25%
정보 품질 20%
날씨·페이스·위치 각 10%

음식점·카페

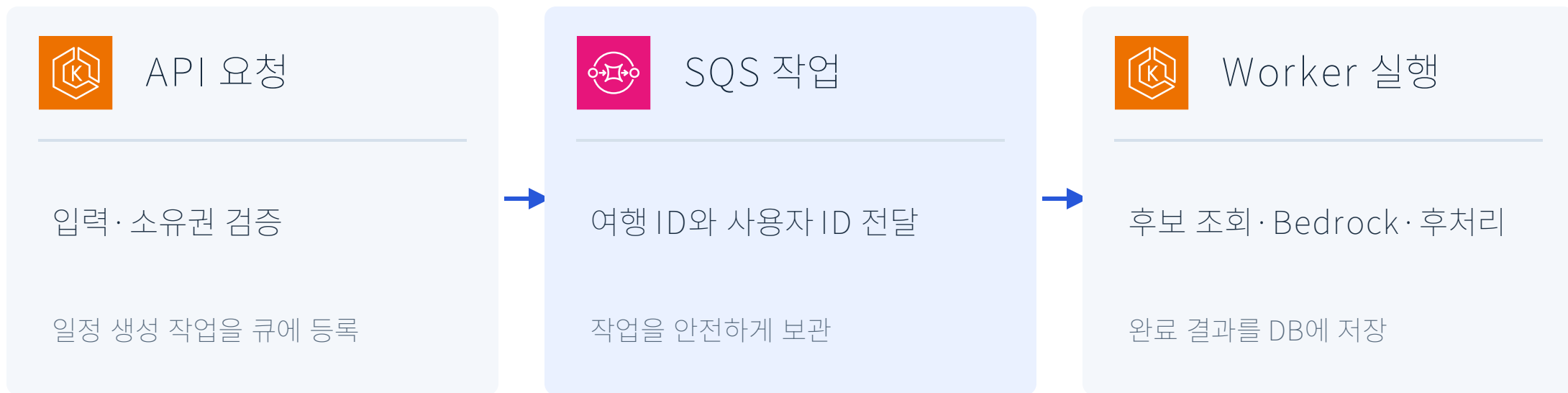
일반 15분 · FOOD 30분

음식점: 평점 55%
카페: 평점 60~65%
거리·지역성·경험 점수 반영

가중치 의미

후보 제거 비율이 아니라 후보 간 우선순위를 계산하는 점수 비중

SQS로 긴 일정 생성 작업을 분리합니다



생성 상태

PENDING → PROCESSING → COMPLETED / FAILED

같은 이미지라도 API와 Worker를 역할별로 실행하고, Bedrock 호출은 Worker에서만 허용합니다.

AI 결과를 검증하고 실패 시 서버 일정으로 전환합니다



Bedrock의 역할

선별된 후보 ID만 받아
일자별 방문 순서를 제안

서버 검증

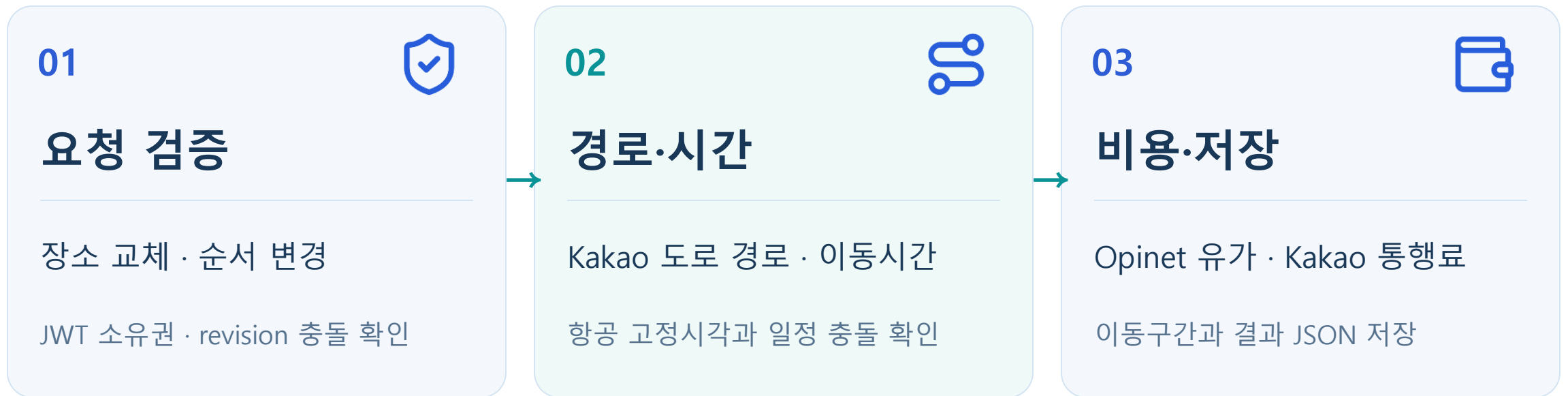
- 01 여행 일자 dayNumber가 빠짐없이 있는가
- 02 장소 ID와 유형이 후보 풀에 존재하는가
- 03 같은 장소가 중복되지 않았는가
- 04 필수 목적지와 여행 일자가 모두 포함됐는가

Fallback 조건

Bedrock 오류 · JSON 파싱 실패 · 후보 ID 검증 실패 시
서버 점수와 시간대 규칙으로 대체 일정을 구성

일정 편집 후 경로와 이동비를 다시 계산합니다

사용자 변경을 검증한 뒤, 변경된 날짜만 다시 계산합니다



예상 이동비 $\text{주행거리} \div \text{차량 연비} \times \text{Opinet 지역 유가} + \text{Kakao 통행료}$

데이터의 변화 속도에 맞춰 TTL을 달리했습니다

같은 조건의 재조회는 줄이고, 일정에 영향을 주는 변화는 빠르게 반영합니다

빠른 변화 반영

15분



도로 경로

교통 상황·소요시간

1시간



Opinet 지역 유가

지역·유종별 유가

2시간



단기 날씨

가까운 시간대의 예보

반복 조회 재사용

30분



장소 후보 4종

숙소·음식점·카페·관광지

6시간



중기 날씨

같은 지역·날짜 예보

12시간



항공편 운항 후보

같은 출발 조건의 후보



Redis 장애 시

API Pod의 메모리 캐시가 최근 결과를 최대 60초 임시 제공

캐시당 최근 256개만 보관해 Pod 메모리 사용량 제한

JWT 인증과 사용자별 여행 소유권 검증



로그인

BCrypt 비밀번호 검증
Access Token 발급
Refresh Token 발급

토큰 보관

Access: 응답 본문
Refresh: HttpOnly Cookie
선택적 Redis 저장

API 보호

Authorization Bearer 검증
사용자 ID로 여행 소유권 확인
편집 시 revision 충돌 검사

Refresh Token 회전과 로그아웃 폐기는 `JWT_REFRESH_STORE_ENABLED=true`일 때 Redis와 연동

Spring Security의 Stateless 정책으로 모든 보호 API에 JWT 인증 적용

03

AWS ARCHITECTURE

AWS 아키텍처

임승준

03

CLOUD SYSTEM

AI 생성은 오래 걸리고, 같은 조회는 반복됐습니다

백엔드에서 확인한 문제



긴 AI 생성 대기

Bedrock 호출이 포함된 일정 생성은
일반 조회보다 처리시간이 김



운영 환경에 반영한 해결책

API와 Worker 분리

일반 조회와 긴 작업을 나누고
생성 요청은 큐에서 대기



같은 정보의 반복 호출

장소 변경 시 경로·날씨 등을 재조회
같은 조건의 호출과 대기가 반복

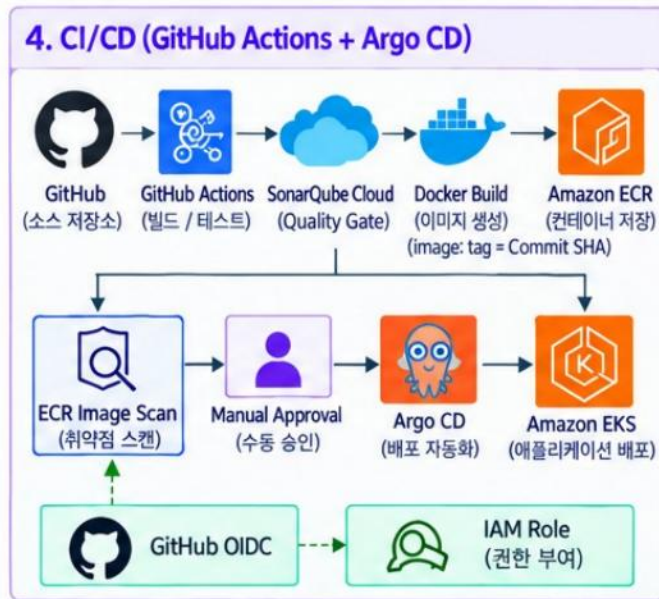
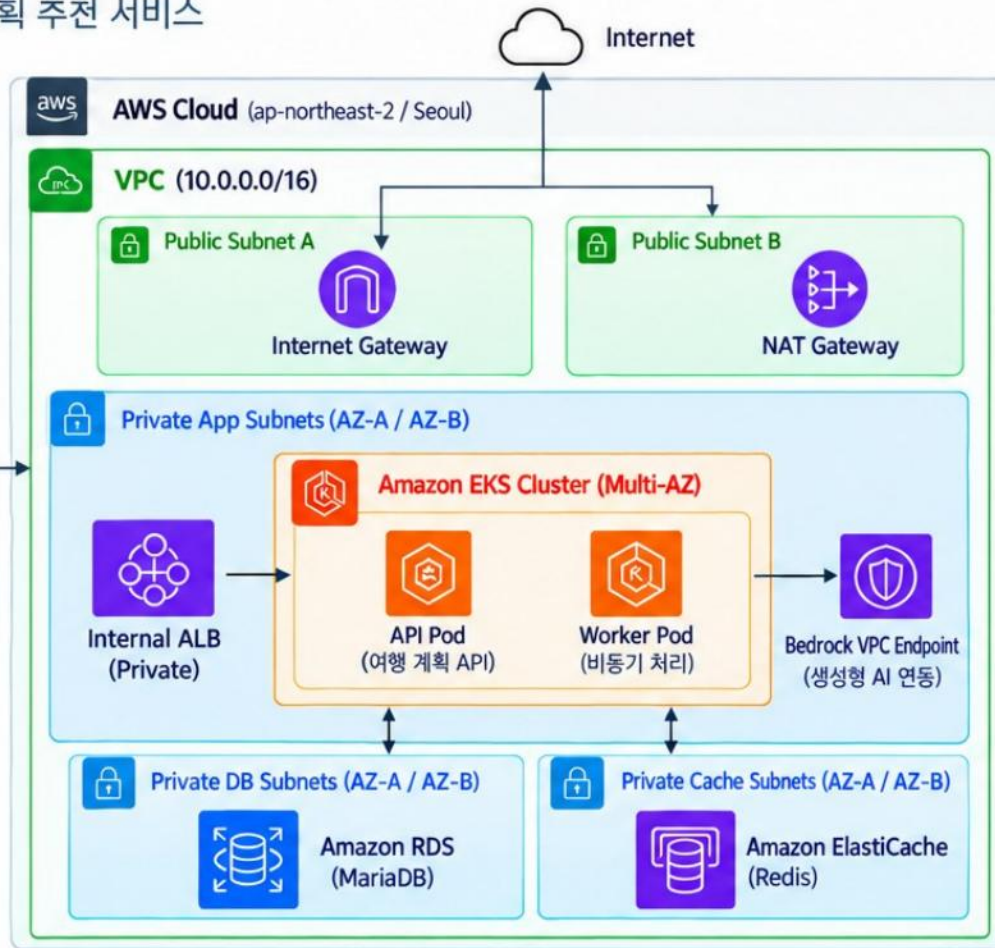


캐시 정책을 AWS에 적용

백엔드의 TTL 기준을 유지하며
ElastiCache에서 결과 재사용

AI 여행 플래너 (AI Travel Planner)

AWS 기반의 AI 여행 계획 추천 서비스



서비스 전체를 세 계층으로 나눴습니다

01 진입·보호 계층

Route 53 · CloudFront · WAF

정적 화면 전달과 API 진입 분리



02 애플리케이션 계층

ALB · EKS API / Worker · SQS

일반 요청과 AI 생성 작업 분리



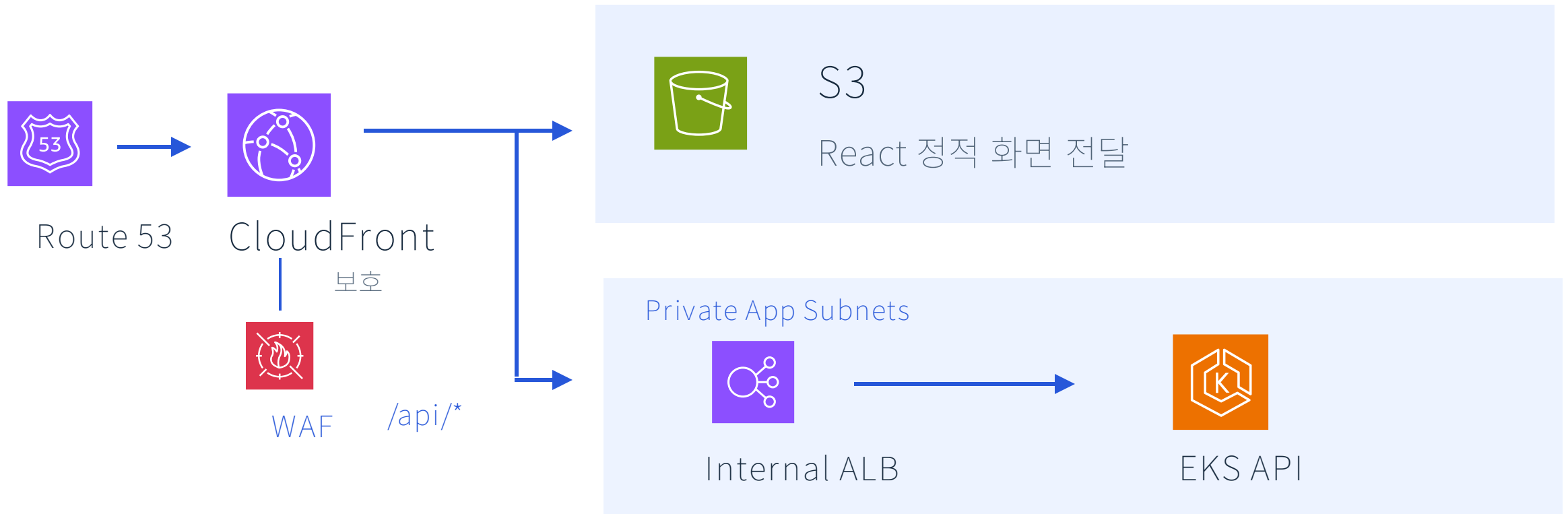
03 데이터 계층

RDS MariaDB · ElastiCache

영구 저장과 반복 조회 캐시 분리

앱·DB·캐시: 사설 서브넷 분리 / 외부 API: NAT / Bedrock: VPC Endpoint

화면과 API를 연결하고, 내부 서버를 보호합니다



일정 생성 요청은 큐를 거쳐 Worker로 전달됩니다



실행·접근 경계

서로 다른 ServiceAccount와 APP_RUNTIME_ROLE을 사용

API와 생성 작업을 별도 실행 단위로 분리하고, 역할에 맞게 접근 권한을 제한합니다.

SQS를 선택한 기준

긴 AI 생성 작업을 Worker로 넘기는 방법을 처리 방식과 운영 부담으로 비교했습니다.

필요했던 기능

요청 급증을 큐에서 완충하고
실패 작업을 다시 처리

Amazon SQS

관리형 작업 큐

대안 비교

Kafka

스트리밍·재생

이번 요청형 작업에는 범위가 큼

RabbitMQ

복잡한 라우팅

브로커 운영이 추가됨

SQS

관리형 작업 큐

현재 요구에 맞춤

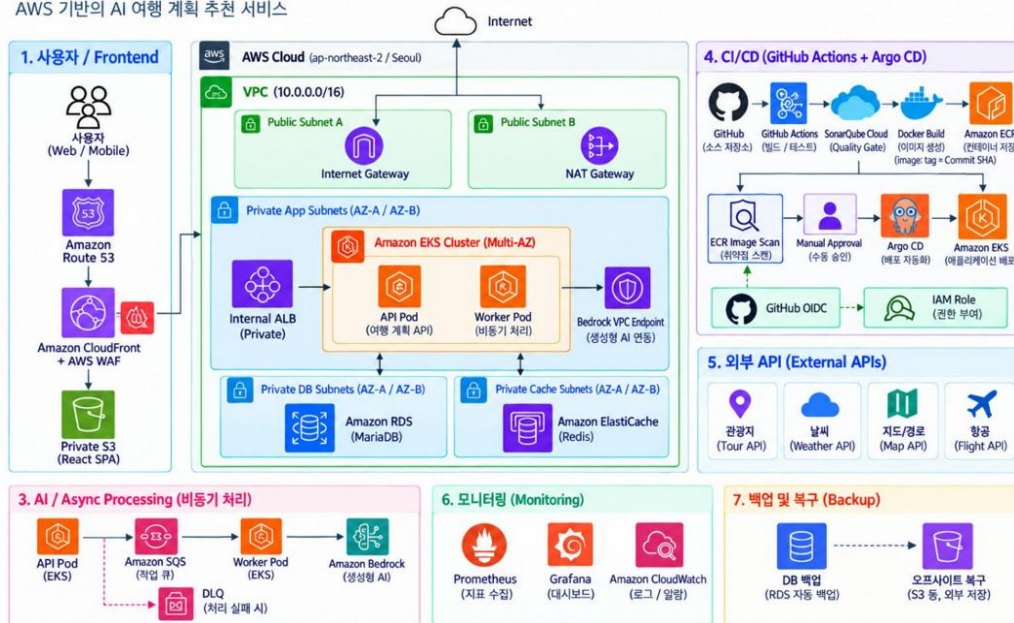
결론 요청 1건을 Worker에 넘기는 현재 패턴에는 SQS가 적합했습니다.

불필요한 호출과 상시 자원을 줄인 구조

전체 아키텍처에서 비용이 발생하는 경로를 분리했습니다

AI 여행 플래너 (AI Travel Planner)

AWS 기반의 AI 여행 계획 추천 서비스



01 CloudFront + S3

정적 화면을 앱 서버 밖에서 제공
API Pod 부하 감소

02 Redis + TTL

경로·날씨·장소 조회 결과 재사용
외부 API·DB 호출 감소

03 SQS + Worker

긴 생성 작업을 큐에 모아 처리
Worker 자원 효율화

04 HPA / Scale-in

부하가 낮아지면 Pod 수 축소
상시 실행 자원 최소화

접근 권한 제한과 복원 가능성 확인

권한을 나누고, 백업은 실제 복구 가능 여부까지 점검합니다

01 접근 권한 제한

IAM



API·Worker·운영 콘솔의 접근 권한을
역할 단위로 분리

Secrets Manager



민감정보와 접속 정보를 코드 외부에서 관리

02 복원 가능성 확인

DB 백업

일일 백업과 스냅샷을 보관

복구 절차

복원 절차와 오프사이트 백업 가능 여부를 점검

운영 기준 필요한 역할만 접근하고, 백업 후에는 복원 가능성까지 확인

TripBuddy AWS 인프라 설계의 결론

일반 요청과 AI 작업, 저장소, 접근 권한, 복구 범위를 구분했습니다

01

요청과 AI 작업

일반 요청은 API가 처리
AI 생성은 Worker로 분리

02

비동기 작업

SQS로 생성 요청을 보관
요청 증가에 대응

03

저장소 구분

Redis는 반복 조회에 사용
RDS는 최종 일정 보관

04

권한과 복구

역할별 접근 권한 분리
백업 후 복원 가능성 점검

장애 영향 범위 · 확장성 · 보안 · 운영 편의성 · 비용 효율성을 함께 고려

TripBuddy

04

CI / CD

CI/CD · 운영 · 모니터링

최승민

04

DELIVERY FLOW

검증한 변경을 사용자에게 반영하는 배포 흐름

FRONTEND



BACKEND



유튜버의 소개 직후, 접속이 한꺼번에 몰린다면



“평소의 Pod 두 개로도 버틸 수 있을까?”

몰린 요청을 처리 성격에 따라 나눴습니다



정적 화면



CloudFront + S3

앱 서버를 거치지 않고 제공



일반 API



ALB + EKS API

조회·수정·상태 확인 처리



AI 일정 생성



SQS + Worker + Bedrock

처리 가능한 속도로 큐에서 소비



반복 조회



Redis Cache

동일 조건의 결과 재사용

Pod 두 개를 고정하자 CPU 부담이 커졌습니다

3,000 RPS

목표 요청률

약 2,289 QPS

실제 처리량

약 261 ms

평균 응답시간



API Pod CPU: Request 대비 약 170~180%

CPU Request 0.25 core 기준 · CPU Limit 초과나 오류율을 뜻하는 수치가 아님

HPA 기반 Auto Scaling으로 고부하 대응

부하 전 2개



HPA 목표 70%



최대 설정 6개

```
C:\Users\CloudDX>kubectl get hpa,pods -n travel -o wide
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/travel-api	Deployment/travel-api	cpu: 1%/70%	2	6	6	4m7s

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
pod/rds-tunnel	1/1	Running	0	7d23h	10.0.20.147	ip-10-0-20-73.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-9dccm	1/1	Running	0	6d	10.0.10.102	ip-10-0-10-182.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-b8frh	1/1	Running	0	96s	10.0.20.138	ip-10-0-20-73.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-f87hp	1/1	Running	0	6d	10.0.20.47	ip-10-0-20-73.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-l7mnp	1/1	Running	0	96s	10.0.10.107	ip-10-0-10-182.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-mlxqj	1/1	Running	0	81s	10.0.10.78	ip-10-0-10-182.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-snrtf	1/1	Running	0	81s	10.0.20.38	ip-10-0-20-73.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-worker-568dd4656c-bwvrr	1/1	Running	0	6d	10.0.20.130	ip-10-0-20-73.ap-northeast-2.compute.internal	<none>	<none>

```
C:\Users\CloudDX>
```

Pod 수 3배로 자동 확장

API Pod 6개 모두 Ready 1/1

기존 Pod 2개와 새 Pod 4개 확인 · 캡처 시점의 1%는 확장을 유발한 순간의 CPU 값이 아님

API Pod 6→2개, CPU Request 배정량 66.7% 감소

트래픽 감소 → CPU 사용률 감소 → Scale-in

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
pod/rds-tunnel	1/1	Running	0	7d23h	10.0.20.147	ip-10-0-20-73.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-9dccm	1/1	Running	0	6d1h	10.0.10.102	ip-10-0-10-182.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-api-85cfc779f7-l7mnp	1/1	Running	0	13m	10.0.10.107	ip-10-0-10-182.ap-northeast-2.compute.internal	<none>	<none>
pod/travel-worker-568dd4656c-bwvrr	1/1	Running	0	6d1h	10.0.20.130	ip-10-0-20-73.ap-northeast-2.compute.internal	<none>	<none>

C:\Users\CloudDX>

API Pod 6 → 2개 · 4개 축소 (−66.7%)

CPU Request 1.5 → 0.5 vCPU · 1.0 vCPU 감소

실제 AWS 비용 절감액은 미측정 · Node 수·인스턴스 종류·축소 유지 시간 확인 필요

TripBuddy

05

마무리

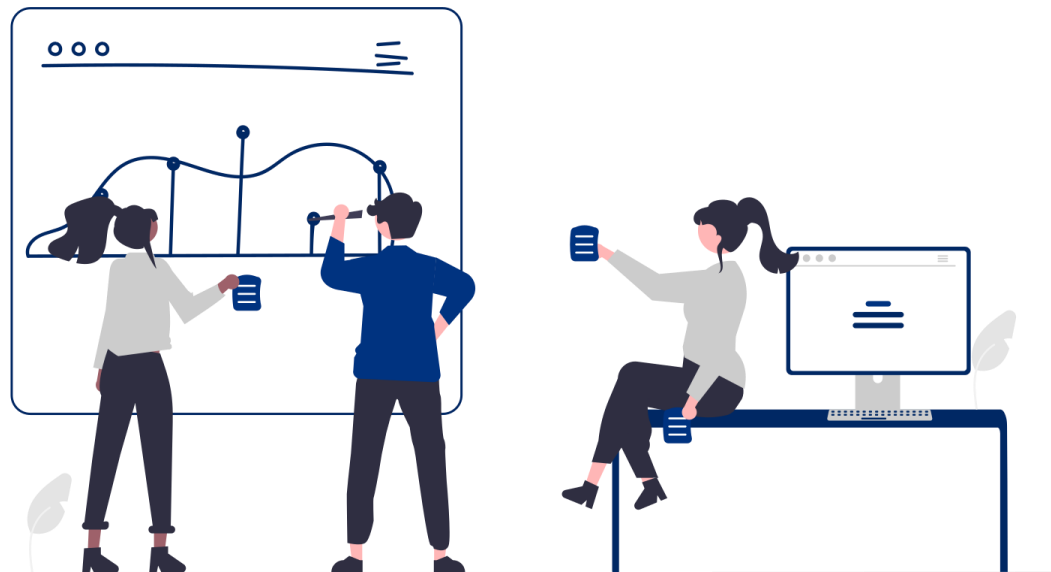
프로젝트 경험

05

REFLECTION

서버 구축을 넘어, 하나의 실무 프로젝트로

기획부터 운영 검증까지 팀이 함께 서비스의 전체 흐름을 경험했습니다



아이디어를 실제 서비스로 만들고,
배포 후 동작까지 확인했습니다.

각 파트가 같은 목표를 공유하며
화면과 API, 클라우드 환경을 하나로 연결했습니다.

01 기획·요구사항

→ 02 개발·연동

→ 03 AWS 배포

→ 04 운영 검증

CI/CD 담당 공백 대응

작업의 순서와 일정을 체계적으로 관리했기에, 갑작스러운 공백에도 유연하게 대처할 수 있었습니다.

판단 기준

팀이 공유한 프로젝트 WBS



작업 일정과 담당, 선행 관계를 함께 관리

01

담당 공백 발생

CI/CD 담당 팀원의 취업으로
배포 업무 인계가 필요해졌습니다.

02

우선순위 조정과 업무 인수

WBS로 남은 작업과 선행 관계 확인
우선순위와 담당 재조정
PM이 CI/CD 업무 인수

03

배포 작업 지속

담당 공백을 메우고
배포 작업을 이어갔습니다.

우리는 아이디어를
구현하는 데서
멈추지 않았습니다.

의견이 다를 때는 사용자 경험을 기준으로 조율하고,
계획이 달라질 때도 함께 방향을 다시 잡았습니다.

그렇게, 함께 끝까지 완성한
하나의 서비스가 TripBuddy입니다.



함께 쌓아 올린 하나의 결과



TEAM TripBuddy

함께 만든 여행

서로 다른 역할, 하나의 서비스

AI 일정 생성부터 변경 후 재조율까지

이시우

Frontend

여행 조건 입력과 일정 편집 화면

서정찬

Backend

실제 후보 기반의 AI 일정 생성

임승준

AWS Architecture

서비스 실행 환경과 운영 기반

최승민

PM · CI/CD

기획·협업 조율과 자동 배포

TripBuddy

여행 준비의 부담을 줄이고,
내가 원하는 여행에 집중하도록.

